PRASAD V. POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

(Autonomous | NBA Accredited | NAAC A+ Accredited)

CSE (AI & ML)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & ML)

OBJECT-ORIENTED PROGRAMMING THROUGH JAVA

23AM3302

II B Tech – I Semester

Academic Year: 2026 – 27

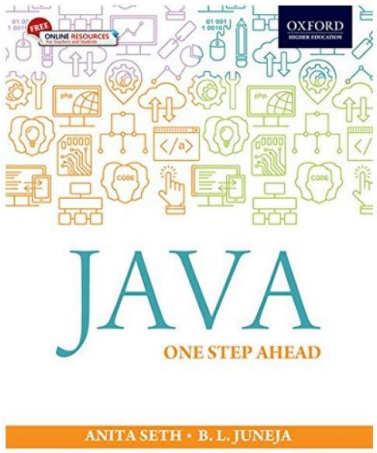
Course Co-ordinator

Dr Suresh Yadlapati

Department of CSE (AI & ML)



- Syllabus
- Micro Syllabus
- Question Bank



Text Books

- JAVA one step ahead, Anitha Seth, B.L.Juneja, Oxford.
- Joy with JAVA, Fundamentals of Object Oriented Programming, Debasis Samanta, Monalisa Sarma, Cambridge, 2023.

JAVA FUNDAMENTALS

1. OBJECT ORIENTED PROGRAMMING (OOP)

A. Basic Concepts
- Class
- Object
- Method
- Message Passing

B. Principles
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

C. Advantages of OOP
- Modularity
- Reusability
- Maintainability
- Flexibility

2. PROGRAM STRUCTURE IN JAVA

1. Introduction to Java
2. Writing Simple Java Programs
3. Elements / Tokens in Java Programs
   - Keywords
   - Identifiers
   - Literals
   - Operators
   - Separators
4. Java Statements
5. Command Line Arguments
6. User Input to Programs
7. Escape Sequences
8. Comments
   - Single Line (//)
   - Multi Line (/* */)
   - Documentation (/** */)
9. Programming Style
   - Indentation
   - Naming Conventions
   - Readability
   - Documentation

3. DATA TYPES, VARIABLES AND OPERATORS

A. Data Types

1. Primitive (8 Types)
- byte
- short
- int
- long
- float
- double
- char
- boolean

2. Non-Primitive (Reference Types)
- String
- Array
- Class
- Interface

B. Variables
- Declaration
- Scope of Variable
- Type Casting
- Static Variables
- Static Methods
- final Attribute

C. Constants
- Literal Constants (e.g., 10, 3.14, 'A')
- Symbolic Constants (using final keyword)

D. Formatted Output
printf() Method

E. Operators

1. Assignment (=)
2. Arithmetic (+, −, *, /, %)
3. Increment (++)
4. Decrement (−−)
5. Ternary (? :)
6. Relational (<, >, <=, >=, ==, !=)
7. Boolean Logical (&&, ||, !)
8. Bitwise Logical (&, |, ^, ~, <<, >>, >>>)
9. Precedence & Associativity (of Operators)

4. CONTROL STATEMENTS

A. Decision Making
- if Expression
- Nested if Expressions
- if–else Expressions
- Ternary Operator (? :)
- Switch Statement

B. Iteration / Looping
- while Expression
- do–while Loop
- for Loop
- Nested for Loop
- For–Each for Loop

C. Jump Statements
- break Statement
- continue Statement

These topics form the foundation of Java Programming and are essential for building problem-solving skills using the Java language.

JAVA – CLASSES, OBJECTS, METHODS & STRING HANDLING

1. CLASSES AND OBJECTS

1 Introduction
- Basic idea of Classes and Objects in Java

2 Class Declaration and Modifiers
- Class Declaration
- Modifiers: public, default, protected, private, final, abstract

3 Class Members
- Data Members (Fields)
- Member Methods
- Constructors
- Blocks (Instance, Static)

4 Declaration of Class Objects
- Creating Objects
- Using Objects (Accessing Members)

5 Assigning One Object to Another
- Copying Object References

2. CONSTRUCTOR AND METHODS

1 Introduction
- Importance of Methods and Constructors

2 Defining Methods
- Syntax
- Method Call
- Return Types (void / value)

3 Overloaded Methods
- Same name, different parameter lists

4 Constructors
- Default Constructor
- Parameterized Constructor

5 Overloaded Constructor Methods
- Same class name, different parameter lists

6 Class Objects as Parameters in Methods
- Passing Objects to Methods

7 Access Control
- private
- default
- protected
- public

8 Accessing Private Members of Class
- Using public methods (Getters/Setters)

9 Recursive Methods
- Method calling itself

10 final Method
- Prevents Method Overriding

11 Passing Arguments by Value and by Reference
- Call by Value
- Call by Reference

12 Keyword this
- Using this to refer current object
- this() – Constructor Chaining

13 final and static
- final Variable
- final Class
- static Variable
- static Method
- static Block

14 Nested Classes
- Static Nested Class
- Inner Class
- Local Inner Class
- Anonymous Inner Class

3. STRING HANDLING IN JAVA

1 Introduction
- Importance of String Handling

2 Interface Char Sequence
- Common interface for String classes

3 Class String
- Creating String Objects
- String Immutability
- String Literals vs String Objects

4 Methods for Extracting Characters from Strings
- charAt()
- getChars()
- substring()
- toCharArray()

5 String Operations
- Comparison: equals(), equalsIgnoreCase(), compareTo(), compareToIgnoreCase()
- **Modifying:** concat(), replace(), trim(), toLowerCase(), toUpperCase()
- **Searching:** indexOf(), lastIndexOf(), contains(), startsWith(), endsWith()

6 Class String Buffer
- Creating StringBuffer Objects
- Commonly Used Methods (append(), insert(), delete(), reverse(), capacity(), length(), etc.)
- Mutable Character Sequence

Summary

These topics build the foundation of Object Oriented Programming in Java, including the creation and use of classes and objects, implementation of methods and constructors with access control and other features, and efficient handling of strings using the String and StringBuffer classes.

JAVA – ARRAYS, INHERITANCE & INTERFACES

1. ARRAYS

1. Introduction
2. Declaration and Initialization of Arrays
3. Storage of Array in Computer Memory
4. Accessing Elements of Arrays
5. Operations on Array Elements
   - Traversal, Insertion, Deletion, Update
6. Assigning Array to Another Array
7. Dynamic Change of Array Size
8. Sorting of Arrays
   - Bubble Sort, Selection Sort, etc.
9. Search for Values in Arrays
   - Linear Search, Binary Search
10. Class Arrays
    - java.util.Arrays (Methods and Uses)
11. Two-dimensional Arrays
12. Arrays of Varying Lengths (Jagged Arrays)
13. Three-dimensional Arrays
14. Arrays as Vectors
    - Concept of Vector using Arrays

2. INHERITANCE

1. Introduction
   - Concept and Importance of Inheritance
2. Process of Inheritance
3. Types of Inheritances
   - Single, Multilevel, Hierarchical, Multiple, Hybrid
4. Universal Super Class – Object Class
   - java.lang.Object
5. Inhibiting Inheritance of Class Using Final
   - final Class
6. Access Control and Inheritance
   - public, protected, default, private
7. Multilevel Inheritance
8. Application of Keyword super
   - Accessing Superclass Members
9. Constructor Method and Inheritance
   - Calling Superclass Constructor
10. Method Overriding
    - Rules and Example
11. Dynamic Method Dispatch
    - Runtime Polymorphism
12. Abstract Classes
    - Abstract Class and Abstract Methods

3. INTERFACES

1. Introduction
   - Concept and Importance of Interfaces
2. Declaration of Interface
   - Syntax and Members
3. Implementation of Interface
   - Using implements Keyword
4. Multiple Interfaces
   - A Class Implementing Multiple Interfaces
5. Nested Interfaces
   - Interface Inside Interface
6. Inheritance of Interfaces
   - Interface Extending Another Interface
7. Default Methods in Interfaces
   - default Keyword and Usage
8. Static Methods in Interface
   - static Keyword and Usage

SUMMARY

These topics cover advanced concepts of Java including Arrays for data handling, Inheritance for code reusability and hierarchy, and Interfaces for abstraction and multiple inheritance using interfaces. Together they strengthen OOP design and implementation in Java.

JAVA – PACKAGES, EXCEPTION HANDLING & I/O

1. PACKAGES AND JAVA LIBRARY

1. **Introduction**
   - Concept and Importance of Packages
2. **Defining Package**
   - Syntax and Example
3. **Importing Packages and Classes into Programs**
   - import statement, static import
4. **Path and Class Path**
   - CLASSPATH Environment Variable
5. **Access Control**
   - public, protected, default, private
6. **Packages in JAVA SE**
   - Overview of commonly used packages
7. **java.lang Package and its Classes**
   - Automatically imported package
   - Commonly used classes
8. **Class Object**
   - Methods of java.lang.Object class
9. **Class Math**
   - Common Math Methods
10. **Wrapper Classes**
    - Integer, Double, Boolean, Character, etc.
11. **Auto-boxing and Auto-unboxing**
    - Converting primitive to objects
    - Converting objects to primitive

2. EXCEPTION HANDLING

1. **Introduction**
   - Need and Importance of Exception Handling
2. **Hierarchy of Standard Exception Classes**
   - Throwable → Exception → RuntimeException
3. **Keywords throws and throw**
   - throws : Declaring Exceptions
   - throw : Generating Exceptions
4. **try, catch, and finally Blocks**
   - Basic structure and flow
5. **Multiple Catch Clauses**
   - Handling different exceptions
6. **Class Throwable**
   - Methods of class Throwable
7. **Unchecked Exceptions**
   - Runtime Exceptions
8. **Checked Exceptions**
   - Compile-time Exceptions
9. **Custom Exceptions**
   - Creating User-defined Exceptions

3. JAVA I/O AND FILE

1. **Java I/O API**
   - Overview of java.io and java.nio packages
2. **Standard I/O Streams**
   - System.in, System.out, System.err
3. **Types of Streams**
   - Byte Streams
   - Character Streams
4. **Byte Streams**
   - InputStream, OutputStream and their subclasses
5. **Character Streams**
   - Reader, Writer and their subclasses
6. **Scanner Class**
   - Reading input from keyboard, files, etc.
7. **Files in Java: File Class**
   - Creating File objects
   - Common methods (exists, createNewFile, delete, length, etc.)
8. **FileInputStream and FileOutputStream Classes**
   - Reading from and writing to files

SUMMARY: This section covers important Java concepts: organizing code with Packages and using Java Library classes, handling runtime errors using Exception Handling, and performing input/output operations and file handling using Java I/O API. These topics are essential for building robust and well-structured Java applications.

JAVA – MULTITHREADED PROGRAMMING & COLLECTIONS FRAMEWORK

1. MULTITHREADED PROGRAMMING

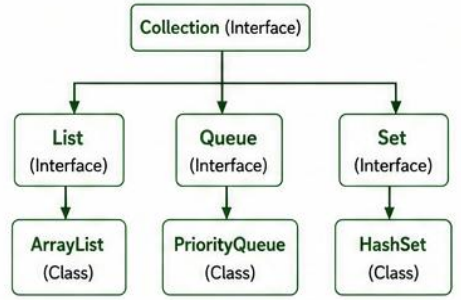
1. **Introduction**
   - Concept and Overview of Multithreading
2. **Need for Multiple Threads**
   - Improves CPU utilization
   - Better responsiveness, resource sharing
3. **Multithreaded Programming for Multi-core Processor**
   - Enables parallel execution of threads
   - Better performance on multi-core systems
4. **Thread Class**
   - java.lang. Thread class
   - Important constructors and methods
5. **Main Thread – Creation of New Threads**
   - Main (primary) thread
   - Creating threads using Thread class
6. **Thread States**
   - New, Runnable, Blocked, Waiting,
   - Timed Waiting, Terminated
7. **Thread Priority**
   - Priority levels (MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY)
   - getPriority() and setPriority() methods

2. COLLECTIONS FRAMEWORK

1. **Introduction**
   - Overview of Collections Framework
2. **Purpose of Collection Framework**
   - Provides a unified architecture for representing and manipulating groups of objects
3. **Application of Collection Framework**
   - Data storage, retrieval, searching, sorting, and manipulation
5. **Methods Defined in Collection Interface**
   - add(), addAll(), remove(), removeAll()
   - clear(), contains(), containsAll()
   - size(), isEmpty()
   - iterator()
   - toArray()
6. **Interface Iterator**
   - Used to traverse elements of a collection
   - hasNext(), next(), remove()
   - Fail-fast behavior

Hierarchy of Collection Interfaces / Classes

- Root Interface: Collection



7. **Collection Classes/Interfaces**
   - List – Ordered collection (allows duplicates)
   - Queue – Holds elements prior to processing
   - Set – Unordered collection (no duplicates)
8. **Important Classes**
   - ArrayList – Resizable array implementation of List
   - HashSet – Hash table based implementation of Set
   - PriorityQueue – Priority based queue implementation

SUMMARY: Multithreaded Programming allows concurrent execution of tasks, improving performance and resource utilization, especially in multi-core systems. The Collections Framework provides ready-to-use classes and interfaces to store, retrieve, manipulate, and process groups of objects efficiently.

ATLMS

Active Teaching Learning Methodologies

1. FLIPPED CLASSROOM



Students learn concepts at home through videos/ resources and apply them in class through discussions and activities.

2. QUIZ



Short quizzes used to assess understanding, reinforce learning and encourage active participation.

3. CASE STUDY



Real-life scenarios analyzed by students to identify issues, evaluate options and propose solutions.

4. PEER TEACHING



Students teach and explain concepts to their peers, promoting deeper understanding and collaboration.

5. PROBLEM-BASED LEARNING (PBL)



Students work in groups to solve real-world problems, develop critical thinking and apply their knowledge.

SEMESTER READINESS PROGRAM

FLIPPED CLASSROOM ACTIVITY

UNIT-I: JAVA FUNDAMENTALS

Topics: OOP Concepts | Java Program Structure | Data Types & Variables | Operators | Control Statements



STEP 1: BEFORE CLASS (STUDENT PREPARATION)

Learning Materials

(Shared 2–3 days before class)

- PPT / Notes
- Concept Map

PPT / Notes

Concept Map

Student Task

Each group prepares a 3–5 minute explanation on one topic.

GROUP	TOPIC
Group 1	OOP Principles
Group 2	Java Program Structure
Group 3	Data Types & Variables
Group 4	Operators
Group 5	Control Statements

IN-CLASS ACTIVITIES

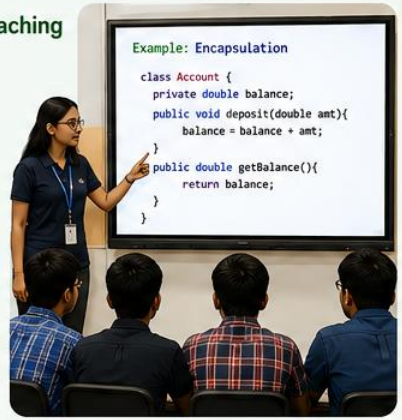
1 Activity 1: Student Teaching

Each group presents:

- ✓ Concept explanation
- ✓ One real-life example
- ✓ One Java code snippet

Examples:

- Encapsulation → ATM Machine
- Inheritance → Vehicle → Car
- Loops → Attendance Processing



2 Activity 2: Faculty Summary and Discussion (10 Minutes)

- Clarification of doubts
- Summary of key concepts
- Feedback on presentations

POST-CLASS ACTIVITIES

Assignment

1. Prepare a Concept Map of Unit-I.
2. Write two conceptual questions with answers.
3. Develop a Java program using:
   - User Input
   - Operators
   - Control Statements

EXPECTED OUTCOMES

- ✓ Understand Java fundamentals effectively.
- ✓ Explain concepts through peer learning.
- ✓ Apply concepts in simple Java programs.
- ✓ Improve communication and presentation skills.

Learn Before Class, Engage in Class, Master After Class!

- KEY FEATURES OF FLIPPED CLASSROOM

Pre-class Preparation | Active In-class, Engagement | Peer Learning & Discussions | Better Understanding & Retention | Improved Skills & Confidence

BEFORE CLASS Prepare & Explore learning materials → **IN CLASS** Discuss, Present, Collaborate & Learn → **AFTER CLASS** Practice, Reflect & Apply

“The best way to learn is to teach, the best way to retain is to apply.”

– Peer Learning, Active Learning

★ SEMESTER READINESS PROGRAM ★

QUIZ–BASED LEARNING ACTIVITY

UNIT–II: CLASSES, OBJECTS, CONSTRUCTORS & STRING HANDLING

LEARN! THINK! COMPETE!

Classes & Objects | Constructors & Methods | Access Control | this, static, final | Nested Classes | String & StringBuffer

BEFORE CLASS

- ✓ Share Unit-II notes / PPT
- ✓ Students prepare important concepts
- ✓ Divide the class into 6–7 teams
- ✓ Bring your thinking... Get ready to compete!

TEAMS (6–7)

TEAM 1 | TEAM 2 | TEAM 3 | TEAM 4 | TEAM 5 | TEAM 6 | TEAM 7

OBJECTIVE

- Test conceptual understanding
- Promote participation and teamwork
- Strengthen analytical & problem-solving skills

DURING CLASS (40–50 MINUTES)



ROUND 1: RAPID FIRE (10 MINUTES)

Sample Questions:

1. What is the purpose of a constructor?
2. Can constructors be overloaded?
3. What is the use of the this keyword?
4. Difference between String and StringBuffer?
5. What is a nested class?

ROUND 2: CODE ANALYSIS (15 MINUTES)

```
class Test {
  int x;
  Test(int x){
    this.x = x;
  }
}
```

Questions:

- What is the role of this?
- Which constructor is invoked?
- What will happen if this is removed?

ROUND 3: IDENTIFY THE OUTPUT (10 MINUTES)

```
String s1 = "Java";
String s2 = "Java";
System.out.println(s1 == s2);
```

Predict the output and justify.

ROUND 4: SCENARIO-BASED QUESTIONS (10 MINUTES)

Example:
A university wants to store student details and provide different ways to create student objects. Which concept should be used and why?

Expected Answer: Constructor Overloading

EXPECTED OUTCOMES

- Explain classes and objects.
- Differentiate constructors and methods.
- Apply access modifiers correctly.
- Analyze String operations.
- Improve logical reasoning and teamwork.

SUCCESS TIPS

- Listen carefully
- Answer confidently
- Discuss with team
- Respect every opinion
- Think logically

Think Code Learn Win!

COMPETE WITH PASSION... LEARN WITH ENTHUSIASM... SUCCEED TOGETHER!

SEMESTER READINESS PROGRAM

CASE STUDY – ATLM ACTIVITY

UNIT–III: ARRAYS, INHERITANCE, INTERFACES





CASE STUDY: COLLEGE MANAGEMENT SYSTEM

PROBLEM STATEMENT

A college wants to automate its management system.
The system should maintain details of:

- Students
- Faculty
- Administrative Staff



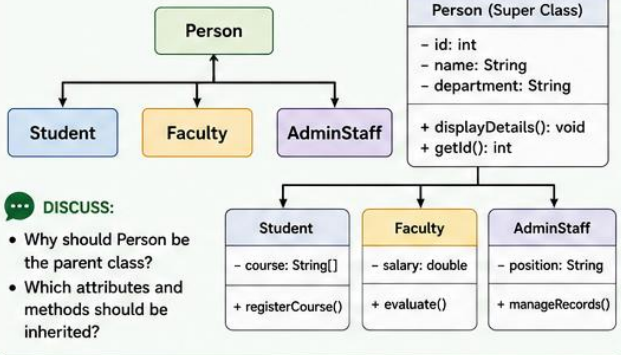
All these share common information:

- ID
- Name
- Department

However, each category has additional features.

PART 1: IDENTIFY INHERITANCE HIERARCHY

Create a superclass:



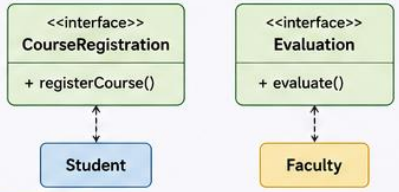
DISCUSS:

- Why should Person be the parent class?
- Which attributes and methods should be inherited?

PART 2: APPLY INTERFACES

REQUIREMENTS:

- Students can register for courses.
- Faculty can evaluate students.





QUESTIONS:

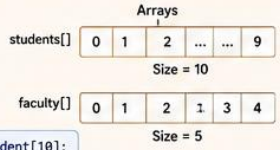
1. Why use interfaces instead of classes?
2. Can a class implement multiple interfaces?

PART 3: USE ARRAYS

Store details of:

- 10 Students
- 5 Faculty members

Using arrays:




```
Student students[] = new Student[10];
Faculty faculty[] = new Faculty[5];
```

QUESTIONS:

1. Why are arrays useful here?
2. What happens if the number of students exceeds array size?



GROUP ACTIVITY (BY THE STUDENTS)

- Design Inheritance Hierarchy
- Identify Interfaces
- Array Usage
- Advantages of OOP Design
- Presentation & Conclusion



EXPECTED OUTCOME

- Design class hierarchies using inheritance
- Differentiate between abstract classes and interfaces
- Apply arrays for object storage
- Analyze real-world systems using OOP concepts
- Improve logical reasoning and teamwork



REAL-WORLD PROBLEM → College Management System

OOP CONCEPTS APPLIED → Inheritance, Interfaces, Arrays

ANALYSIS & DISCUSSION

SOLUTION & PRESENTATION

LEARNING & APPLICATION

SEMESTER READINESS PROGRAM

PEER TEACHING ACTIVITY

UNIT–IV: PACKAGES, EXCEPTION HANDLING & JAVA I/O



Packages & Import Statements	Wrapper Classes & Auto-boxing	Exception Handling	Custom Exceptions	Java I/O & File Handling

STEP 1: GROUP FORMATION (BEFORE CLASS)

Divide the class into 5 groups.

GROUP	TOPIC
Group 1	Packages and Import Statements
Group 2	Wrapper Classes & Auto-boxing
Group 3	Exception Handling
Group 4	Custom Exceptions
Group 5	Java I/O and File Handling

- Provide study material 2–3 days in advance.

STEP 2: PREPARATION BY STUDENTS

Each group prepares:

1. **Concept Explanation** (3 minutes)
   Example: Group 3 explains:
   - Checked Exceptions
   - Unchecked Exceptions
   - try-catch-finally



2. **Real-Life Example** (2 minutes)
   Example:
   ATM Withdrawal → Insufficient Balance → Exception Occurs → Handled Gracefully
3. **Java Program Demo** (3 minutes)
   Example:

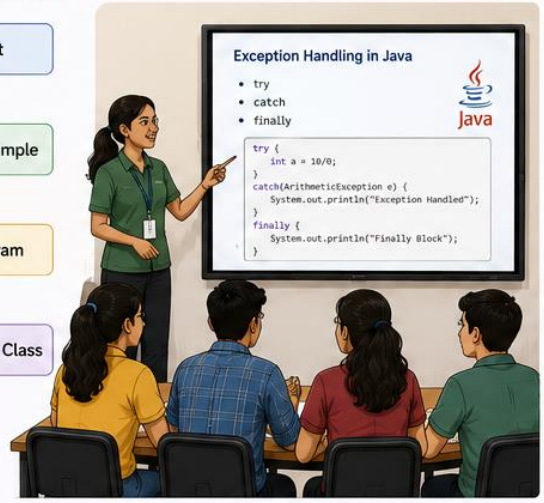
```
try {
    int a = 10/0;
}
catch(ArithmeticException e) {
    System.out.println("Exception Handled");
}
```

STEP 3: PEER TEACHING SESSION (25–30 MINUTES)

Each group teaches the remaining students.

Presentation Format:

Concept → Real-life Example → Java Program → Questions to Class



STEP 4: PEER QUESTIONING (10 MINUTES)

After each presentation, students ask questions such as:

Packages
- Why do we use packages?
- Difference between import and package?

Exceptions
- Why is exception handling required?
- Difference between throw and throws?

I/O
- Difference between Byte Stream and Character Stream?
- When should Scanner be used?

STEP 5: REFLECTION ACTIVITY (5 MINUTES)

Each student writes:

1. One concept learned.
2. One concept still confusing.

EXPECTED OUTCOMES

Students will:

- Explain Java packages and libraries.
- Differentiate checked and unchecked exceptions.
- Demonstrate exception handling using try-catch-finally.
- Use Java I/O classes for file operations.
- Improve communication and collaborative learning skills.



SEMESTER READINESS PROGRAM

ATLM: PROBLEM-BASED LEARNING (PBL)

UNIT–V: MULTITHREADING & COLLECTIONS FRAMEWORK



PROBLEM STATEMENT

Online Food Delivery System

A food delivery company wants to develop a system to manage customer orders.

REQUIREMENTS

1. Multiple customers place orders simultaneously.
2. Orders should be processed independently.
3. The system should maintain all orders efficiently.
4. The system should prioritize urgent orders.

CONCEPTS COVERED

MULTITHREADING

- Thread Creation
- Thread States
- Thread Priority
- Concurrent Execution

COLLECTIONS FRAMEWORK

- ArrayList
- HashSet
- PriorityQueue
- Iterator

STEP 1: PROBLEM ANALYSIS (10 MINUTES)

Ask students:

What happens if 100 customers place orders at the same time?

Expected Discussion:

- Single-threaded execution causes delays.
- Multiple threads can process orders simultaneously.

STEP 2: GROUP FORMATION

GROUP	RESPONSIBILITY
Group 1	Thread Design
Group 2	ArrayList Usage
Group 3	PriorityQueue Usage
Group 4	Order Processing Logic
Group 5	Presentation & Testing

STEP 3: SOLUTION DESIGN

Order Class

```
class Order
{
   int orderId;
   String customerName;
   int priority;
}
```

Thread Class

```
class OrderThread extends Thread
{
   public void run()
   {
      System.out.println("Processing Order");
   }
}
```

Collections

```
ArrayList<Order> orders;
PriorityQueue<Order> urgentOrders;
HashSet<String> uniqueCustomers;
```

STEP 4: STUDENT ACTIVITIES

THREAD USAGE

- Why should each order be a separate thread?
- What is the role of thread priority?

COLLECTION SELECTION

REQUIREMENT	COLLECTION
Store Orders	ArrayList
Remove Duplicates	HashSet
Urgent Orders	PriorityQueue

STEP 5: PRESENTATION (15 MINUTES)

Each group presents:

1. Problem Analysis
2. Selected Java Concepts
3. Proposed Solution
4. Benefits



EXPECTED OUTCOMES

- Explain the need for multithreading.
- Create and manage threads.
- Select appropriate collection classes.
- Apply PriorityQueue, ArrayList, and HashSet in real-world problems.
- Develop problem-solving and teamwork skills.

SUPPORTING CONCEPTS (USED)

- Thread Creation & States
- Thread Priority
- Concurrent Execution
- Collections Framework
- Iterator (for traversal)

CUSTOMERS PLACE ORDERS → MULTIPLE THREADS PROCESS ORDERS CONCURRENTLY → ORDERS STORED USING COLLECTIONS → URGENT ORDERS PRIORITIZED → ORDERS DELIVERED EFFICIENTLY → CUSTOMER SATISFACTION

Course Outcomes

Upon successful completion of the course, students will be able to:		
CO1	Describe the fundamental concepts of object-oriented programming and core Java programming constructs	L2
CO2	Apply object-oriented principles to implement Java programs	L3
CO3	Use packages, exception handling, file I/O, multithreading, and the Collections Framework to build Java applications	L3
CO4	Analyze complex Java programs by leveraging advanced features and critically assessing their performance and limitations.	L4

Program Outcomes

1. Engineering Knowledge
2. Problem Analysis
3. Design/Development of Solutions
4. Conduct Investigations of Complex Problems
5. Engineering Tool Usage
6. The Engineer and the World
7. Ethics
8. Individual and Collaborative Team Work
9. Communication
10. Project Management and Finance
11. Life-long Learning

BLOOM'S TAXONOMY – LEVELS OF LEARNING

Bloom's Taxonomy is a hierarchy of cognitive skills used to classify educational learning objectives.

LEVEL	DESCRIPTION	ACTION WORDS (Examples)	EXAMPLES
6 **CREATE** (Creating)	Put elements together to form a new whole; create new ideas or products.	Design, Construct, Plan, Produce, Invent, Develop, Compose, Create	• Design a new machine • Write a novel • Create a new app
5 **EVALUATE** (Evaluating)	Make judgments based on criteria and standards through checking and critiquing.	Evaluate, Judge, Assess, Critique, Defend, Compare, Conclude, Justify	• Evaluate a research paper • Judge the quality of a project • Justify your opinion
4 **ANALYZE** (Analyzing)	Break information into parts to explore understandings and relationships.	Analyze, Examine, Compare, Contrast, Investigate, Categorize, Differentiate	• Analyze a poem • Compare two theories • Examine causes of a problem
3 **APPLY** (Applying)	Use information in new situations by applying knowledge, facts, methods, etc.	Apply, Use, Solve, Demonstrate, Implement, Practice, Operate	• Apply a formula • Use principles to solve problems • Demonstrate a skill
2 **UNDERSTAND** (Understanding)	Explain ideas or concepts by interpreting, summarizing, paraphrasing, etc.	Explain, Summarize, Interpret, Describe, Classify, Paraphrase, Discuss	• Explain a concept • Summarize a paragraph • Describe a process
1 **REMEMBER** (Remembering)	Recall facts and basic concepts without necessarily understanding them.	Remember, List, Define, Tell, Identify, Recall, Recognize	• List the types of cells • Define a term • Identify the capital of India

E-Resources & MOOCs for Learning Java

- **Coursera** and **edX** offer university-certified Java tracks from global elite institutions.
- **Udemy** delivers comprehensive, industry-expert-led practical video masterclasses.
- **University of Helsinki MOOC** provides top-tier free courses with instant, automated grading.
- **Codecademy** and **JetBrains Academy** offer hands-on, browser-based, and project-driven practice.
- **NPTEL** and **Swayam** deliver free, high-quality technical lectures from top IIT professors.
- **Interview & Reference Hubs**: **LeetCode**, **HackerRank**, and **GeeksforGeeks** provide coding challenges and extensive Java documentation.